

Web Technology

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Web Technology — One-Liners	3
Web Fundamentals	3
HTML Basics and Elements	3
HTML Forms	3
CSS	4
XML	4
XML Processing: DOM vs SAX	4
DHTML	5
JavaScript	5
AJAX	5
VBScript	5
Java Beans	5
EJB	6
ASP	6
JSP	6
Servlets	6
JDBC	7
COM / DCOM / CORBA	7
PHP Basics	7
PHP Strings and Operators	7
PHP Control Structures	7
PHP Arrays and Functions	8
PHP Forms, Sessions, Files	8
PHP Sessions, Errors, Filters, DB	8

Web Technology — One-Liners

Rapid-revision, double-layered facts for the IBPS SO (IT Officer) exam. Every line pairs a structural metric (how many / which port / what size / which method) with the specific descriptive items. Numbering is continuous across the whole syllabus.

Web Fundamentals

1. The **WWW** was invented in **1989** by **Tim Berners-Lee** at **CERN**, running as one service on top of the **Internet**.
2. The default port for **HTTP** is **80** while **HTTPS** uses port **443** (HTTP over **TLS/SSL**).
3. **HTTP** is a **stateless, application-layer** protocol, so state is added externally via **cookies, sessions**, and **URL rewriting**.
4. There are **5 main URL parts**: **scheme, host, port, path**, and **query** (plus an optional **fragment**).
5. **DNS** runs on port **53** and acts as the "**phonebook of the Internet**," mapping **domain names** to **IP addresses**.
6. The **6 common HTTP methods** are **GET, POST, PUT, DELETE, HEAD**, and **PATCH**.
7. HTTP status code **200** means **OK**, **301** means **Moved Permanently**, **304** means **Not Modified**.
8. HTTP status code **404** means **Not Found** (client error) while **500** means **Internal Server Error** (server error).
9. HTTP responses fall into **5 classes**: **1xx** info, **2xx** success, **3xx** redirection, **4xx** client error, **5xx** server error.
10. The **3 popular web servers** are **Apache, Nginx**, and **Microsoft IIS**.
11. Chrome's rendering engine is **Blink** with the **V8** JS engine, while Firefox uses **Gecko** with **SpiderMonkey**.
12. **Client-side** technologies (**HTML, CSS, JavaScript, VBScript**) run in the **browser**, while **server-side** ones (**PHP, JSP, ASP, Servlets**) run on the **server**.
13. A **static** page shows fixed content to all users, whereas a **dynamic** page is generated per request from a **database** or script.

HTML Basics and Elements

14. An HTML **element** has **3 parts**: a **start tag**, **content**, and an **end tag**, while an **attribute** sits in the start tag as **name="value"**.
15. The **3 structural tags** of an HTML document are **head, body**, and the enclosing **html**, preceded by the **DOCTYPE** declaration.
16. HTML provides **6 heading levels**, from **h1** (largest) down to **h6** (smallest).
17. The common **void (empty) elements** with no closing tag are **br, hr, img, input, meta**, and **link**.
18. The **2 semantic emphasis tags** are **strong** (importance) and **em** (emphasis), versus the **2 visual tags** **b** (bold) and **i** (italic).
19. The anchor tag **a** uses the **href** attribute for the link and **target** for where it opens, e.g. **_blank** for a new tab.
20. HTML has **3 list types**: **unordered (ul)**, **ordered (ol)**, and **definition (dl)** which uses **dt** and **dd**.
21. An ordered list **ol** supports **5 type values**: **1, A, a, I**, and **i**, plus a **start** attribute.
22. A table cell merges **horizontally** with **colspan** and **vertically** with **rowspan**.
23. The **2 confusable table attributes** are **cellspacing** (gap between cells) and **cellpadding** (space inside a cell).
24. A table's **3 grouping sections** are **thead, tbody**, and **tfoot**, with **caption** for the title.
25. The **img** tag is **void** and uses **src** for the file and **alt** for alternative/accessibility text.
26. HTML5 multimedia uses **2 tags**, **audio** and **video**, each taking a **source** child and a **controls** attribute.
27. Frames use **3 tags**: **frameset** (with rows/cols), **frame**, and **iframe**, where **frameset/frame** are **removed in HTML5** but **iframe** stays valid.
28. A **frameset** document has **no body** element, since the **frameset** replaces it.

HTML Forms

29. A **form** has **3 key attributes**: **action** (target URL), **method** (GET/POST), and **enctype** (encoding).
30. The **2 form methods** are **GET** (data in the URL, ~**2048** char limit, cached, bookmarkable) and **POST** (data in the body, large, not cached).
31. The **default** form method is **GET**, while **POST** is used for sensitive or large submissions.

32. **Radio buttons** with the same **name** allow **only one** selection, whereas **checkboxes** allow **multiple**.
33. The classic **input types** include **text**, **password**, **radio**, **checkbox**, **submit**, **reset**, **file**, and **hidden**.
34. HTML5 added input types such as **email**, **number**, **date**, **range**, **color**, **tel**, and **url**.
35. The **3 multi-option controls** are **select** (dropdown), **textarea** (multi-line), and grouping with **fieldset** plus **legend**.
36. A file-upload form needs **method="POST"** and **enctype="multipart/form-data"** to work.

CSS

37. **CSS** stands for **Cascading Style Sheets** and is maintained by the **W3C** to separate **presentation** from **content**.
38. A CSS rule has **2 parts**: a **selector** and a **declaration block** of **property: value** pairs.
39. There are **3 ways** to apply CSS: **inline** (style attribute), **internal** (style tag), and **external** (link to a .css file).
40. Among placements, **inline CSS** has the **highest** priority while **external CSS** is best for **maintainability**.
41. The **basic selectors** are **universal (*)**, **element**, **class (.)**, and **id (#)**.
42. The **combinator selectors** are **descendant (space)**, **child (>)**, **adjacent sibling (+)**, and **general sibling (~)**.
43. **Pseudo-classes** use one colon like **:hover**, while **pseudo-elements** use two colons like **::before**.
44. The CSS **box model** has **4 layers** from inside out: **content**, **padding**, **border**, and **margin**.
45. **Padding** is the space **inside** the border while **margin** is the space **outside** the border between elements.
46. With default **content-box**, total width = **width + 2×padding + 2×border + 2×margin**, but **border-box** makes width include padding and border.
47. The **5 position values** are **static** (default), **relative**, **absolute**, **fixed**, and **sticky**.
48. The common **display values** include **block**, **inline**, **inline-block**, **none**, **flex**, and **grid**.
49. CSS specificity weights are **inline=1000**, **id=100**, **class=10**, and **element=1**.
50. In the cascade, priority order is **!important > inline > id > class > element**, and ties break by **source order** (last wins).

XML

51. **XML** stands for **eXtensible Markup Language**, is **case-sensitive**, and uses **user-defined** tags to **store/transport** data.
52. An XML document begins with a **prolog** declaration: **version="1.0"** and **encoding="UTF-8"**.
53. **Well-formed** XML needs **1 root element**, all tags **closed**, **proper nesting**, and **quoted** attribute values.
54. **Valid** XML is **well-formed PLUS** conforms to a **DTD** or **XSD**.
55. XML differs from HTML in **3 ways**: XML is **extensible**, **case-sensitive**, and **data-focused**, while HTML is fixed-tag and presentation-focused.
56. The **5 predefined XML entities** are **<**, **>**, **&**, **'**, and **"**.
57. **DTD** stands for **Document Type Definition**, can be **internal or external**, but has **no data types** and is **not written in XML**.
58. DTD declarations use **3 keywords**: **ELEMENT**, **ATTLIST**, and **ENTITY**.
59. **XSD** stands for **XML Schema Definition**, is **written in XML**, and **supports data types and namespaces**, making it more powerful than DTD.
60. The XSD root element is **schema** (often prefixed **xs:**), unlike DTD which uses **DOCTYPE**.

XML Processing: DOM vs SAX

61. **DOM** stands for **Document Object Model** and loads the **entire** document into memory as a **tree**, allowing **random access** and **modification**.
62. **SAX** stands for **Simple API for XML**, is **event-based** (a **push parser**), reads **sequentially**, and is **read-only**.
63. **DOM** is **memory-heavy** and better for **small** files, while **SAX** uses **low memory** and is better for **large** files.
64. SAX fires **events** such as **startDocument**, **startElement**, **characters**, **endElement**, and **endDocument**.
65. Between the two, only **DOM** allows **modifying** the document, while **SAX** offers **no random access**.
66. **StAX** is a **pull parser**, contrasting with SAX which is a **push parser**.
67. **XSLT** transforms XML into **HTML/XML/text**, and it relies on **XPath** to navigate nodes.

68. **XPath** uses **path expressions** to **select nodes** within an XML document.

DHTML

69. **DHTML** stands for **Dynamic HTML** and is **NOT** a single language but a **combination** of technologies.

70. **DHTML** combines **4 components**: **HTML**, **CSS**, **JavaScript**, and the **DOM**.

71. In DHTML, the **DOM** is the **glue** that lets **JavaScript** access and change **HTML** and **CSS**.

72. DHTML is **event-based**, updating content **without** a server round-trip.

JavaScript

73. **JavaScript** was created by **Brendan Eich** at **Netscape** in **1995** and is standardized as **ECMAScript (ECMA-262)**.

74. JavaScript is **interpreted**, **loosely typed**, **case-sensitive**, and **client-side** (also server-side via **Node.js**).

75. JavaScript has **7 primitive types**: **Number**, **String**, **Boolean**, **Undefined**, **Null**, **Symbol**, and **BigInt**, plus the **Object** type.

76. The quirky result of **typeof null** is **"object"**, a long-standing JavaScript bug.

77. JavaScript has **3 variable declarations**: **var** (function scope), **let** (block scope), and **const** (block, constant).

78. The key comparison difference is **==** (value with coercion) versus **===** (strict value **and** type).

79. JavaScript loops include **4 kinds**: **for**, **while**, **do-while**, plus **for-in** (keys) and **for-of** (values).

80. JavaScript functions come in **3 forms**: **declaration**, **expression**, and **arrow (=>)**, where declarations are **hoisted**.

81. The core built-in objects include **Math**, **Date**, **String**, **Array**, **JSON**, and **RegExp**.

82. The **BOM** provides **5 objects**: **window**, **navigator**, **screen**, **location**, and **history**.

83. The **3 dialog functions** are **alert()**, **prompt()**, and **confirm()**.

84. The **5 DOM selection methods** include **getElementById**, **getElementsByClassName**, **getElementsByTagName**, **querySelector**, and **querySelectorAll**.

85. To create and attach nodes, JavaScript uses **createElement** and **appendChild**, and reads/writes content via **innerHTML**.

86. Returning **false** from **onsubmit** **cancel**s a form submission during client-side validation.

87. Common JavaScript events include **onclick**, **onload**, **onmouseover**, **onchange**, **onsubmit**, and **onkeydown**.

88. The modern way to bind events is **addEventListener("event", handler)**.

89. Client-side validation improves **speed/UX** but is **not secure**, so validation must repeat on the **server**.

AJAX

90. **AJAX** stands for **Asynchronous JavaScript And XML** and the term was coined by **Jesse James Garrett** in **2005**.

91. AJAX allows **partial** page updates **without** a full reload, using the **XMLHttpRequest (XHR)** object as its core.

92. The XHR object's **2 main methods** are **open(method, url, async)** and **send()**.

93. The XHR **readyState** has **5 values (0–4)**, where **4** means **DONE**.

94. A successful AJAX call is when **readyState == 4 AND status == 200**.

95. The XHR readyState values are **0 UNSENT**, **1 OPENED**, **2 HEADERS_RECEIVED**, **3 LOADING**, and **4 DONE**.

96. Modern AJAX transfers **JSON** (called **AJAJ**) and uses the **fetch()** API instead of raw XHR.

97. AJAX's **2 main drawbacks** are **SEO** difficulty and a broken **back button**.

VBScript

98. **VBScript** is a **Microsoft** language based on **Visual Basic**, is **case-insensitive**, and runs client-side **only in Internet Explorer**.

99. The **2 client-side scripting languages** historically were **JavaScript** (all browsers) and **VBScript** (IE only).

100. **VBScript** also runs **server-side** in **ASP** and in the **Windows Script Host**, but it is now **deprecated**.

Java Beans

101. A **JavaBean** is a **reusable** software component requiring a **public no-argument constructor** and **getter/setter** methods.

- 102. A JavaBean must implement **Serializable** to support **persistence** of its state.
- 103. JavaBeans expose **boolean** properties with an **isX()** getter instead of **getX()**.
- 104. There are **4 JavaBean property types**: **simple**, **indexed**, **bound**, and **constrained**.
- 105. A **bound** property **notifies** listeners via **PropertyChangeListener**, while a **constrained** property can be **vetoed** via **VetoableChangeListener**.
- 106. A vetoed change throws a **PropertyVetoException** from the **constrained** property mechanism.
- 107. **BDK** stands for **Bean Development Kit**, whose test container was the **BeanBox**.
- 108. The JavaBeans API lives in the **java.beans** package and includes **BeanInfo**, **Introspector**, and **PropertyDescriptor**.
- 109. **Introspection** analyzes a bean's **properties, methods, and events** using **reflection** and **BeanInfo**.
- 110. The **3 JavaBean design-time features** are **introspection**, **customization**, and **persistence**.

EJB

- 111. **EJB** stands for **Enterprise Java Beans**, runs **server-side** inside an **EJB container**, and is part of **Java EE / Jakarta EE**.
- 112. There are **3 EJB types**: **Session**, **Entity**, and **Message-Driven (MDB)** beans.
- 113. **Session beans** come in **3 flavors**: **stateless**, **stateful**, and **singleton**.
- 114. **Entity beans** are **deprecated** since **EJB 3.0** and replaced by **JPA** entities.
- 115. A **Message-Driven Bean (MDB)** is **asynchronous** and listens to **JMS** messages.
- 116. The **EJB container** manages **4 cross-cutting concerns**: **transactions**, **security**, **concurrency**, and **pooling**.
- 117. A **JavaBean** is **client-side/single-JVM** while an **EJB** is **server-side/distributed/container-managed**.

ASP

- 118. **ASP** stands for **Active Server Pages**, is a **Microsoft** server-side technology, and runs on the **IIS** web server.
- 119. ASP files use the **.asp** extension and use **VBScript** (default) or **JScript** for scripting.
- 120. ASP's built-in objects include **Request**, **Response**, **Session**, **Application**, and **Server**.
- 121. The successor to classic ASP is **ASP.NET**, using **.aspx** files on the **.NET** framework.

JSP

- 122. **JSP** stands for **Java Server Pages**, a **server-side** technology that is **translated into a Servlet** by the container.
- 123. The JSP lifecycle methods are **3**: **jspInit()**, **_jspService()**, and **jspDestroy()**.
- 124. Among JSP lifecycle methods, **_jspService()** **cannot** be overridden, while **jspInit()** and **jspDestroy()** can.
- 125. JSP has **9 implicit objects**: **request**, **response**, **out**, **session**, **application**, **config**, **pageContext**, **page**, and **exception**.
- 126. The JSP **out** implicit object is of type **JspWriter**, and **exception** is available only on an **isErrorPage="true"** page.
- 127. JSP scripting has **3 elements**: **scriptlet** (**<% %>**), **expression** (**<%= %>**), and **declaration** (**<%! %>**).
- 128. A JSP **expression** outputs a value and takes **no semicolon**, while a **declaration** defines instance variables/methods.
- 129. JSP has **3 directives**: **page**, **include**, and **taglib**.
- 130. JSP defines **4 scopes**: **page**, **request**, **session**, and **application** (narrowest to widest).
- 131. JSP standard actions include **jsp:useBean**, **jsp:include**, **jsp:forward**, and **jsp:setProperty**.
- 132. The first JSP request is **slower** because of one-time **translation** and **compilation** into a servlet.

Servlets

- 133. A **Servlet** is a **server-side Java class** running in a **web container** that uses **threads** (not processes) for efficiency.
- 134. The **2 servlet packages** are **javax.servlet** (generic) and **javax.servlet.http** (HTTP-specific).
- 135. The servlet lifecycle has **3 methods**: **init()**, **service()**, and **destroy()**.
- 136. Among lifecycle methods, **init()** and **destroy()** run **once**, while **service()** runs on **every** request.
- 137. The **service()** method dispatches to **doGet()** or **doPost()** based on the HTTP method.

- 138. A servlet container creates **1 instance** served by **multiple threads**, so instance variables are **not thread-safe**.
- 139. **JSDK** stands for **Java Servlet Development Kit**, and **web.xml** is the **deployment descriptor**.
- 140. The base HTTP servlet class is **HttpServlet** and the protocol-independent one is **GenericServlet**.
- 141. There are **4 session-tracking techniques**: **cookies**, **URL rewriting**, **hidden form fields**, and **HttpSession**.
- 142. A server-side session is obtained with **request.getSession()** returning an **HttpSession**.
- 143. In Jakarta EE 9+, the servlet package was renamed from **javax.servlet** to **jakarta.servlet**.
- 144. The key difference is that a **servlet** is **Java with HTML embedded**, while a **JSP** is **HTML with Java embedded**.

JDBC

- 145. **JDBC** stands for **Java Database Connectivity** and lives in the **java.sql** and **javax.sql** packages.
- 146. JDBC defines **4 driver types**, where **Type 1** is the **JDBC-ODBC bridge** and **Type 4** is the **pure-Java thin driver**.
- 147. **Type 4** (thin driver) is the **most used** today, while **Type 1** has been **removed**.
- 148. JDBC has **6 steps**: load driver, get connection, create statement, execute query, process ResultSet, and close.
- 149. JDBC uses **executeQuery()** for **SELECT** and **executeUpdate()** for **INSERT/UPDATE/DELETE**.
- 150. A **PreparedStatement** is **precompiled** and **parameterized**, preventing **SQL injection**, unlike a plain **Statement**.
- 151. The driver is loaded with **Class.forName()** and the connection opened with **DriverManager.getConnection()**.

COM / DCOM / CORBA

- 152. **COM** stands for **Component Object Model**, a **Microsoft binary** standard that is **language-independent** and **Windows-only**.
- 153. Every COM object implements the base interface **IUnknown**, which has **3 methods**: **QueryInterface**, **AddRef**, and **Release**.
- 154. COM is the foundation of **2 technologies**: **OLE** and **ActiveX**.
- 155. **DCOM** stands for **Distributed COM** and extends COM **across a network** using **RPC**.
- 156. **CORBA** stands for **Common Object Request Broker Architecture**, defined by the **OMG**, and is **cross-platform**.
- 157. CORBA's core component is the **ORB (Object Request Broker)**, and interfaces are defined in **IDL**.
- 158. CORBA uses **IIOP (Internet Inter-ORB Protocol)** for communication, with a **stub** on the client and a **skeleton** on the server.
- 159. The key contrast is **COM/DCOM = Microsoft/Windows** versus **CORBA = OMG/cross-platform**.

PHP Basics

- 160. **PHP** is a **recursive acronym** for "**PHP: Hypertext Preprocessor**", originally **Personal Home Page**.
- 161. PHP was created by **Rasmus Lerdorf** in **1994** and is a **server-side**, **interpreted**, **open-source** language.
- 162. PHP code is enclosed in **<?php ?>** tags and each statement ends with a **semicolon (;)**.
- 163. PHP supports **3 comment styles**: **//** and **#** for single line, and **/* */** for multi-line.
- 164. PHP's **2 output constructs** are **echo** (no return, multiple args) and **print** (returns **1**, single arg).
- 165. PHP variables start with **\$**, are **case-sensitive**, and are **loosely typed** with no declaration needed.
- 166. PHP **constants** are defined with **define()** or **const** and use **no \$** sign.
- 167. PHP has **8 data types**: **String**, **Integer**, **Float**, **Boolean**, **Array**, **Object**, **NULL**, and **Resource**.

PHP Strings and Operators

- 168. PHP joins strings with the **dot (.)** operator, **not** the **+** operator.
- 169. PHP **double quotes** interpolate variables and escapes, while **single quotes** are **literal**.
- 170. Key PHP string functions include **strlen()**, **strpos()**, **str_replace()**, **substr()**, **explode()**, and **implode()**.
- 171. PHP's comparison operators differ as **==** (value with juggling) versus **===** (value **and** type).
- 172. PHP 7 added the **spaceship (<=>)** operator and the **null-coalescing (??)** operator.

PHP Control Structures

- 173. PHP conditionals number **4**: **if**, **if-else**, **elseif**, and **switch**.

- 174. PHP has **4 loops**: **for**, **while**, **do-while**, and **foreach** (built for arrays).
- 175. A **do-while** loop always executes its body **at least once**, unlike **while**.
- 176. The **foreach** loop iterates arrays as **foreach(\$arr as \$key => \$value)**.

PHP Arrays and Functions

- 177. PHP has **3 array types**: **indexed** (numeric keys from **0**), **associative** (named keys), and **multidimensional**.
- 178. Common PHP array functions include **count()**, **sort()**, **array_push()**, **array_merge()**, and **in_array()**.
- 179. PHP functions support **default arguments**, **pass by reference (&)**, and **variadic (...)** parameters, and names are **case-insensitive**.

PHP Forms, Sessions, Files

- 180. The **9 PHP superglobals** are **\$_GET**, **\$_POST**, **\$_REQUEST**, **\$_SESSION**, **\$_COOKIE**, **\$_FILES**, **\$_SERVER**, **\$_ENV**, and **\$GLOBALS**.
- 181. Form data arrives via **\$_GET** (URL) or **\$_POST** (body), and combined in **\$_REQUEST**.
- 182. File-upload data arrives in **\$_FILES** (not **\$_POST**) and requires **enctype="multipart/form-data"**.
- 183. The **\$_FILES** array exposes **5 keys**: **name**, **tmp_name**, **size**, **type**, and **error**.
- 184. PHP saves an uploaded file with **move_uploaded_file()** from its **tmp_name**.
- 185. PHP file modes include **r** (read), **w** (write/truncate), and **a** (append), where **w** erases existing content.
- 186. PHP file functions include **fopen()**, **fread()**, **fwrite()**, **fclose()**, and **file_get_contents()**.
- 187. PHP email is sent with the **mail(\$to, \$subject, \$message, \$headers)** function.

PHP Sessions, Errors, Filters, DB

- 188. PHP **sessions** are stored **server-side**, started by **session_start()**, and accessed via **\$_SESSION**.
- 189. PHP **cookies** are stored **client-side** with a **~4 KB** limit, set by **setcookie()** and read via **\$_COOKIE**.
- 190. The session id travels in a cookie named **PHPSESSID**, and **session_start()** must run **before** any output.
- 191. PHP error levels include **E_ERROR**, **E_WARNING**, **E_NOTICE**, **E_PARSE**, and **E_ALL**.
- 192. PHP exception handling uses **4 keywords**: **try**, **catch**, **throw**, and **finally**.
- 193. A PHP **finally** block **always** executes whether or not an exception is thrown.
- 194. PHP filters split into **2 families**: **validate** (e.g. **FILTER_VALIDATE_EMAIL**) and **sanitize** (e.g. **FILTER_SANITIZE_STRING**).
- 195. PHP data is filtered through **filter_var()** and **filter_input()**.
- 196. **PHP-ODBC** connects via **odbc_connect()** using a **DSN**, then runs queries with **odbc_exec()**.
- 197. PHP database access uses **2 main APIs**: **mysqli** (MySQL-only) and **PDO** (supports **multiple** databases).
- 198. The old **mysql_** extension was **removed in PHP 7**, leaving **mysqli** and **PDO**.
- 199. To stop SQL injection in PHP, always use **prepared statements** with bound parameters.
- 200. PHP **variable variables** use **\$\$** so one variable's value becomes another variable's name.